CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

Domain 2: Claude Models, Prompting & Context Engineering

Exam objectives covered

1. Select Claude models based on capability, latency, and cost trade-offs, including reasoning/effort settings, migration strategy, and model assignment across stages of an agentic workflow.
2. Design system prompts, reusable templates, and guardrails, including role/instruction separation, variable handling, output specifications, prompt versioning, and separating trusted instructions from untrusted user content.
3. Apply prompting techniques—zero-shot, few-shot, chain-of-thought/extended reasoning, example selection, explicit instructions, and structured output—choosing the right technique and knowing when examples outperform additional instructions.
4. Optimize context-window usage and token budgets through prioritization, conversation-history management, summarization/compaction, retrieval-based access to documents, tool-result optimization, subagent context isolation, and long-running session management.
5. Implement prompt reuse through prompt caching, modular prompt architecture, reusable components, version management, Claude Skills with progressive loading, and the separation of static versus dynamic context.

Q1 **A healthcare SaaS company runs a support agent with 50 registered tools and a 12,000-token system prompt, most of which rarely changes. A new architect is asked to reduce both latency and cost without sacrificing capability. Which combination of changes is MOST effective?**

A Switch every request to Opus and leave the prompt structure unchanged

B **Dynamically load only relevant tool definitions per conversation, place stable instructions in a cacheable prefix, and route simple FAQ turns to a faster model while reserving stronger reasoning for complex turns**

C Delete the system prompt to save tokens immediately

D Keep all 50 tool definitions loaded but switch to Haiku for every request regardless of complexity

CORRECT ANSWER **B**

WHY This combines dynamic tool loading, caching of stable content, and complexity-based model routing—three complementary optimizations that reduce cost/latency without losing capability. Upgrading the model alone doesn't fix structural waste, removing the system prompt discards necessary guardrails, and uniformly using the cheapest model regardless of complexity risks quality failures on hard turns.

Q2 **A bank's claims-eligibility agent retrieves policy clauses at runtime and inserts them into the prompt alongside the customer's question. A security review wants to prevent a scenario where a maliciously crafted customer message, or a compromised policy document, could override the agent's intended behavior. What is the BEST combined mitigation?**

A Concatenate all content into one block and trust the model to sort it out

B **Clearly delimit retrieved/user content as untrusted data (not instructions) in the system prompt, explicitly state that only system-role content can issue instructions, and keep retrieved content limited to only the relevant clauses via retrieval rather than injecting entire documents**

C Remove the system prompt so there's nothing to inject around

D Trust all retrieved content unconditionally since it comes from internal sources

CORRECT ANSWER **B**

WHY Combining explicit trust-boundary instructions (data vs. instructions) with targeted retrieval (rather than whole-document injection) addresses both the injection risk and context bloat simultaneously. Undifferentiated concatenation and unconditional trust both increase injection risk, and removing the system prompt eliminates necessary guardrails rather than protecting against injection.

Q3 **An insurance company's underwriting assistant must output a risk score and rationale as valid JSON, generalize well to idiosyncratic internal risk categories not well known to the model, and allow full auditability of how the score was derived. Which prompting strategy BEST satisfies all three requirements together?**

A Zero-shot prompting with a one-line instruction to "return JSON"

B **An explicit JSON schema specification, combined with a few curated few-shot examples demonstrating the company's idiosyncratic categories, plus a chain-of-thought step that surfaces the weighed factors before the final structured answer**

C A single rigid example with no schema and no reasoning steps

D Instructing the model to skip reasoning and just output the final number for speed

CORRECT ANSWER **B**

WHY This combines three techniques to their respective strengths: schema specification for structured output reliability, few-shot examples for idiosyncratic domain categories, and chain-of-thought for auditable reasoning. A one-line zero-shot instruction under-specifies the task, a single rigid example risks overfitting, and skipping reasoning sacrifices the auditability requirement.

Q4 **A manufacturing company migrates its predictive-maintenance assistant from an older Claude model to a newer generation to improve reasoning quality. After migration, engineers report the model sometimes omits the "confidence level" field that was reliably present before. What is the BEST diagnostic and remediation approach?**

A Assume the new model is broken and revert permanently without further investigation

B **Re-validate the prompt against the new model, since model migrations are not guaranteed to preserve legacy prompt behavior; strengthen the output-format instruction (and add a few-shot example) to explicitly require the confidence field**

C Ignore the missing field since it's a minor cosmetic issue

D Switch to Haiku, assuming smaller models are more format-compliant by default

CORRECT ANSWER **B**

WHY Model migrations warrant systematic prompt re-validation, and a missing structured field is a known class of issue addressed by tightening format instructions and/or adding examples. Permanently reverting without diagnosis forfeits the migration's reasoning benefits, ignoring a missing field used by downstream systems is risky, and model size doesn't inherently determine format compliance.

Q5 **A government document-processing pipeline uses a long-running agent that reviews thousands of pages over several hours, calling a search tool that can return full document text for each of up to 50 results. The team notices the agent is slowing down and sometimes losing track of earlier findings. What is the BEST combined fix?**

A Keep returning all 50 full documents per search and let the context window grow indefinitely

B **Limit tool results to ranked snippets with full text fetched only on demand, and periodically checkpoint/compact the agent's progress by summarizing completed findings while preserving key decisions**

C Restart the agent from zero every time it slows down

D Increase verbosity of every tool result to help the agent "remember more"

CORRECT ANSWER B

WHY This combines tool-result optimization (snippets over full documents, fetched on demand) with periodic compaction of long-running session state—both targeted fixes for the described symptoms of context bloat and lost findings. Unbounded full-document retrieval worsens bloat, restarting discards legitimate progress, and increasing verbosity directly worsens the underlying problem.

Q6 **A retail company wants one core system prompt reused across customer email replies, live chat, and a voice-assistant channel, while maintaining consistent guardrails and brand tone, and wants to A/B test a new guardrails wording safely. Which architecture BEST satisfies these goals together?**

A. Write three fully independent system prompts, one per channel, each edited live in production with no version tracking

B. Build a modular prompt with a shared, versioned guardrails/tone component composed with channel-specific variables, roll out the new guardrails wording behind a version flag, and tag outputs with the active version for attribution

C. Hardcode the guardrails into each channel prompt separately and update all three manually whenever wording changes

D. Avoid versioning entirely since prompts rarely need auditing

CORRECT ANSWER B

WHY This combines modular prompt architecture (shared component across channels) with disciplined version management (tracked rollout and output attribution), satisfying consistency, reuse, and safe experimentation simultaneously. Fully independent, untracked prompts risk inconsistency and unauditable changes, manual multi-copy updates invite drift, and skipping versioning blocks safe experimentation and rollback.

Q7 **A telecom billing assistant handles both simple one-line balance inquiries and complex multi-cycle billing disputes within the same conversation. The architect wants to control cost while preserving reasoning quality on hard cases, and wants the prompting approach to match each case's actual difficulty. Which design is BEST?**

A Use Opus with full chain-of-thought for every single message regardless of complexity

B **Classify each turn's complexity; route simple turns to a fast/cheap model with straightforward zero-shot instructions, and route complex multi-cycle disputes to a stronger model with chain-of-thought prompting that walks through each billing cycle**

C Use Haiku for everything and add more explicit instructions to compensate for its lower depth on hard cases

D Use the same fixed prompting technique and model for every turn to simplify the system

CORRECT ANSWER **B**

WHY This combines complexity-based model routing with technique-matching (zero-shot for simple, chain-of-thought for complex), directly aligning cost and reasoning depth with actual task difficulty. Using the strongest model and techniques everywhere wastes resources on simple turns, relying on Haiku plus extra instructions won't substitute for genuine reasoning depth on hard multi-step cases, and a one-size-fits-all policy ignores the complexity variation described.

Q8 **A SaaS company's agent reuses an 8,000-token stable system prompt across millions of requests, and the team wants to minimize cost while also supporting a specialized "advanced analytics" capability used by less than 2% of users, which requires a large block of additional specialized instructions. Which design BEST addresses both needs?**

A. Permanently add the advanced analytics instructions into the main system prompt for all users

B. **Place the stable, shared instructions in a cacheable prefix for all requests, and implement advanced analytics as a separately loaded Skill that only consumes context when that capability is actually invoked**

C. Remove prompt caching since it conflicts with having specialized capabilities

D. Build advanced analytics as a fully separate product with no shared guardrails or branding consistency

CORRECT ANSWER B

WHY Combining prompt caching on the stable shared prefix with progressive loading (Skills) for the rarely-used specialized capability optimizes both the common case (cost via caching) and the rare case (context efficiency via on-demand loading). Permanently bloating every request with rarely-used instructions wastes tokens, caching and Skills are complementary rather than conflicting, and building a fully separate product sacrifices consistency and reuse.

Q9 **A legal tech firm's contract-review assistant occasionally hallucinates clause citations that don't exist in the uploaded contract. Separately, the firm wants the output to always be structured JSON for downstream tooling. Which prompt design addresses BOTH issues together?**

A Ask the model to cite more clauses to increase apparent thoroughness, with no format requirement

B **Instruct the model to only cite clauses present in the provided contract text (stating "not found" otherwise), include a few-shot example demonstrating that refusal behavior, and specify an explicit JSON schema requiring a citation field tied to verifiable source text**

C Remove all citation requirements to avoid the hallucination risk entirely

D Leave the output format open-ended so the model can decide what's best per request

CORRECT ANSWER B

WHY Explicit grounding instructions plus a demonstrating example reduce hallucinated citations, while a defined JSON schema with a citation field addresses the structured-output requirement—together solving both problems. Asking for more citations increases hallucination risk, removing citations entirely abandons a core feature, and an open-ended format undermines the downstream parsing requirement.

Q10 **An architect reviews a long customer-support conversation (120 turns) spanning several days for a logistics client. Context limits are being approached, a promised refund commitment made on turn 5 must never be lost, and the live agent still needs fast responses. Which combined approach is BEST?**

A Keep the full, unabridged 120-turn history forever and accept rising latency

B **Summarize/compact older turns into a concise history while explicitly tagging and persisting the refund commitment as a protected key fact that survives compaction, keeping recent turns verbatim for immediate context**

C Delete the oldest 100 turns with no summary to free up space quickly

D Start an entirely new conversation with no reference to any prior history

CORRECT ANSWER B

WHY This combines conversation summarization/compaction for general history management with explicit protection of a critical fact, satisfying both the context-budget constraint and the requirement that the commitment is never lost. Keeping everything unabridged doesn't scale, blind deletion risks losing the commitment, and starting fresh loses all continuity including the commitment itself.

Q11 **A manufacturing company's QA assistant needs to decide safety-shutdown recommendations (high-stakes, multi-factor judgment, auditable), while a separate feature in the same product classifies routine inventory counts into categories (simple, high-volume, low-stakes). An architect must select models and prompting techniques for both. Which pairing is BEST?**

A Use Haiku with zero-shot prompting for the safety-shutdown decisions, and Opus with chain-of-thought for inventory classification

B **Use Opus (or a strong reasoning model) with chain-of-thought prompting for safety-shutdown decisions to support accuracy and auditability, and Haiku with straightforward zero-shot/few-shot prompting for high-volume, simple inventory classification**

C Use the same model and the same prompting technique for both features to simplify the architecture

D Use Opus for both features to guarantee maximum quality everywhere

CORRECT ANSWER **B**

WHY This matches model capability and reasoning technique to each task's actual complexity and stakes: strong reasoning plus chain-of-thought for high-stakes judgment, and a fast/cheap model with simple prompting for high-volume, low-complexity classification. Swapping the pairing (A) misapplies capability to the wrong task, forcing uniformity ignores real complexity differences, and using Opus everywhere wastes cost/latency on the simple classification task.

Q12 **A government agency wants its Claude-powered assistant to never impersonate officials, even under roleplay framing, and separately wants to ensure that any prompt changes addressing this guardrail are fully auditable and reversible. Which design satisfies both goals?**

A. Add a single unreinforced line about impersonation with no version tracking of prompt changes

B. Write an explicit guardrail that the impersonation restriction applies regardless of framing (roleplay, hypothetical, etc.) with a defined decline-and-redirect behavior, and manage this guardrail as a versioned, reviewed component so changes are tracked and reversible

C. Remove the system prompt so there is no guardrail to bypass

D. Rely on the model's default behavior with no explicit guardrail or version history

CORRECT ANSWER B

WHY Robust guardrail phrasing that anticipates bypass framing, combined with formal version management of that guardrail component, satisfies both the content-safety goal and the auditability/reversibility goal. An unreinforced, untracked line is fragile and unauditable, removing the system prompt eliminates the guardrail entirely, and relying on default behavior with no explicit guardrail is not a defensible design.

Q13 **A SaaS analytics company's natural-language-to-SQL feature performs inconsistently on complex multi-table joins, while performing fine on simple single-table queries. The team also wants to reduce cost by not always using their most expensive model. Which combined fix is BEST?**

A. Use the same model and the same zero-shot prompt for all queries regardless of complexity

B. Classify query complexity; for simple single-table queries use a faster/cheaper model with schema-grounded few-shot examples, and for complex multi-table joins route to a stronger model with chain-of-thought prompting that reasons through the join logic step by step

C. Always use the most expensive model with no few-shot examples to simplify prompting

D. Reduce cost by removing the schema from the prompt entirely

CORRECT ANSWER B

WHY This combines complexity-based model routing with technique-matching (grounded few-shot for straightforward queries, chain-of-thought for compound join logic), addressing both the quality and cost goals together. Uniform treatment ignores the complexity difference, using the expensive model everywhere without examples doesn't fix the root schema-grounding issue, and removing the schema would worsen accuracy for all queries.

Q14 **An architect at an insurance company is designing a multi-agent claims system: an orchestrator agent delegates to a "document extraction" subagent (processes long scanned PDFs) and an "eligibility reasoning" subagent (makes the coverage determination). Which model and context-architecture choices are MOST appropriate?**

A Give all three agents (orchestrator, extraction, eligibility) full shared context and use Opus everywhere

B **Use a fast/cheap model for document extraction (mechanical field-pulling) with its own isolated context for the long PDF content, a strong reasoning model for eligibility determination, and have each subagent return only concise structured results to the orchestrator rather than sharing raw context**

C Merge all three roles into a single agent with one giant prompt containing all raw PDF text at all times

D Use Haiku for eligibility reasoning since speed matters most for customer satisfaction

CORRECT ANSWER B

WHY This matches model strength to each subagent's actual workload (mechanical extraction vs. judgment-heavy reasoning) while using subagent context isolation to keep the orchestrator's context lean via concise structured handoffs. Full shared context with Opus everywhere wastes resources and bloats context, merging all roles into one giant prompt defeats the purpose of specialization and isolation, and using Haiku for the highest-stakes reasoning step risks quality failures on coverage determinations.

Q15 **A multinational bank is designing its overall Claude architecture governance and asks an architect to summarize, in one coherent policy, how model selection, prompt design, prompting technique, context management, and prompt reuse should interact across its enterprise deployment. Which summary BEST reflects sound architectural principles across all five areas?**

A "Always use the most capable model, the longest possible prompt, and resend full context every turn to maximize safety and quality."

B **"Match model capability to task complexity and stakes; separate trusted instructions from untrusted content with clear guardrails; choose prompting techniques (zero-shot, few-shot, chain-of-thought, structured output) based on the specific failure mode; optimize context via prioritization, retrieval, and compaction rather than ever-growing raw history; and maximize reuse through modular components, versioning, caching, and progressive-loading Skills."**

C "Use one fixed model and one fixed prompt for every feature in the enterprise to simplify governance, regardless of task differences."

D "Minimize cost above all else by always choosing the cheapest model and shortest prompt, regardless of task requirements."

CORRECT ANSWER B

WHY This answer integrates all five objective areas coherently: proportional model selection, trust-boundary guardrails, diagnosis-driven technique choice, active context optimization, and systematic reuse/versioning—reflecting genuine trade-off-aware architecture. Maximizing everything (A) ignores cost/latency trade-offs and injection risk, forcing total uniformity (C) ignores real differences across use cases, and minimizing cost unconditionally (D) risks quality and compliance failures on tasks that genuinely require more capability or context.

Q16 **An architect is designing a multi-step agentic research assistant that performs many sequential tool-use round trips per task, uses the effort parameter for reasoning depth, and must have overall spend predictable per task rather than per request. Which combined configuration BEST achieves this?**

A Set max_tokens once at task start and assume it caps the entire multi-step loop, while leaving effort unset

B **Use a task budget to bound the full agentic loop's combined thinking, tool-call, and output spend across however many round trips occur, set effort to a level appropriate to each step's actual difficulty rather than always max, and remember that thinking tokens are billed as output tokens that also count against each request's own max_tokens**

C Rely solely on per-request max_tokens values, since task budgets do not exist as a documented mechanism

D Set effort to max for every step to guarantee the best possible answer regardless of task difficulty or cost

CORRECT ANSWER **B**

WHY Because max_tokens applies per request rather than across an entire agentic loop, a task budget is the documented mechanism for bounding total spend across many round trips, while effort should be tuned to each step's actual reasoning demand since thinking tokens are billed as output and compete with the per-request max_tokens ceiling. Assuming a single max_tokens value caps the whole loop is incorrect, task budgets are a real mechanism, and maximizing effort everywhere wastes cost on steps that don't need deep reasoning.

Q17 A platform team caches a large, stable system prompt and tool-definition block for a customer-support agent, and separately ships a specialized "escalation-drafting" capability as a versioned Claude Skill loaded only when needed. After adding one new tool and publishing an unpinned update to the Skill, cache hit rates drop sharply and the team is confused because "the cached content didn't get smaller." What is the BEST explanation?

A Cached content still occupies the context window and caching only affects what is charged for those tokens, not whether they count or whether they remain valid; cache prefixes are built in a fixed order (tools, then system prompt, then messages) so the new tool invalidates everything after it, and Skills render into the system prompt in a fixed order too, so an unpinned Skill update that changes a rendered description can also invalidate the cached prefix

B Caching failures are always random and cannot be explained by any configuration change

C Adding a tool or publishing a new Skill version can never affect prompt caching, since caching only depends on the user's message content

D The drop must be due to the context window shrinking, since caching reduces the available window size

CORRECT ANSWER A

WHY Both documented mechanics apply simultaneously here: tool definitions sit earliest in the fixed cache-prefix order so adding one invalidates everything downstream, and an unpinned Skill version change can alter what renders into the system prompt (itself part of that fixed order), also invalidating the cache; separately, cached tokens still occupy the window even though their billing is discounted. Caching issues are traceable to specific configuration changes, both tool and Skill changes can affect the cache, and caching does not change the window's size.

Q18 **A legal research assistant retrieves documents via a search tool and must never let fetched text override its instructions, while also needing every final answer in a strict JSON schema for downstream systems. Which combined design BEST addresses both requirements?**

A Insert search results directly into the system prompt, and leave the output format unspecified so the model can decide

B **Deliver fetched documents only inside tool result blocks, explicitly state in the system prompt that tool-returned content is untrusted data that must never override instructions, optionally screen tool output with a lightweight model before it reaches the main agent, and separately enforce a strict JSON schema (with case-insensitive handling of any enum fields) for the final answer via structured outputs**

C Put the organization's own formatting instructions inside the tool result block next to the search content, since tool results are processed first

D Skip schema enforcement entirely since structured outputs cannot coexist with tool-use workflows

CORRECT ANSWER **B**

WHY The documented structural defense against injected content is to deliver untrusted tool output only through tool result blocks with an explicit untrusted-data statement (and optional lightweight screening), which is independent from and fully compatible with enforcing a strict output schema via structured outputs, including case-insensitive handling of enum values whose exact casing isn't guaranteed. Placing search results in the system prompt grants them misplaced authority, injecting your own instructions into a tool result undermines the skepticism that channel is meant to invoke, and structured outputs work fine alongside tool use.